

# COMP3069 Computer Graphics

## Lab 9

Originally prepared by Dr Kristian Spoerer of UNUK

Modified by Prof Sean He of UNNC

### Learning Outcomes

After Lab 9, you are expected to be able to

1. Specify vertex normals correctly and set up VAO and shader variables required for lighting calculations.
2. Recognise that fragment positions in the fragment shader are in the viewport space, therefore an additional variable is required to store the fragment position in world space for lighting calculations.
3. Implement ambient, diffuse and specular and combine them in an implementation of the phong lighting model.
4. Use the phong lighting model to implement three well-known light types (directional, positional, and spot light).
5. Define and use a function in the fragment shader.
6. Recognise the effect of attenuation on per fragment lighting.
7. Demonstrate the ability to program per fragment lighting calculations.
8. Use some useful GLSL functions, `normalize()`, `max()`, `pow()`, `radians()`, `cos()`, `reflect()` etc.
9. Write a fragment shader for per fragment lighting.

# Setup

In the COMP3069 directory, create a new project called

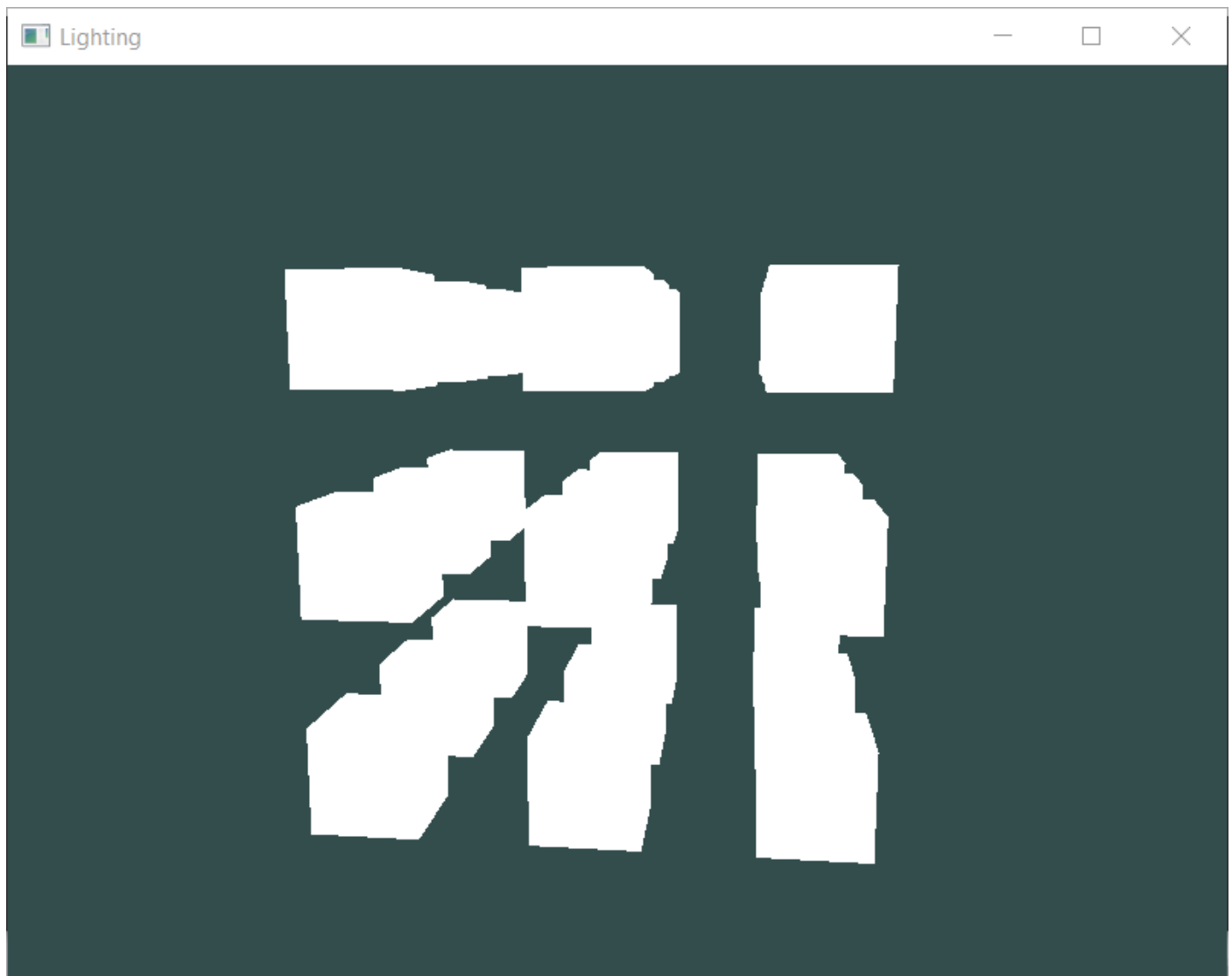
Lab9

Download Lab9.zip from Moodle, unzip the file, and copy the code in the file Lab9.cpp to the Lab9.cpp in your new project.

Copy camera.h, ModelViewerCamera.h, shader.h, util.h, and window.h to your project and add them to the Header Files in the Solution Explorer.

Copy phong.vert and phong.frag to your project, and add them to the Source Files in the Solution Explorer.

If you run the program, then the following window should appear



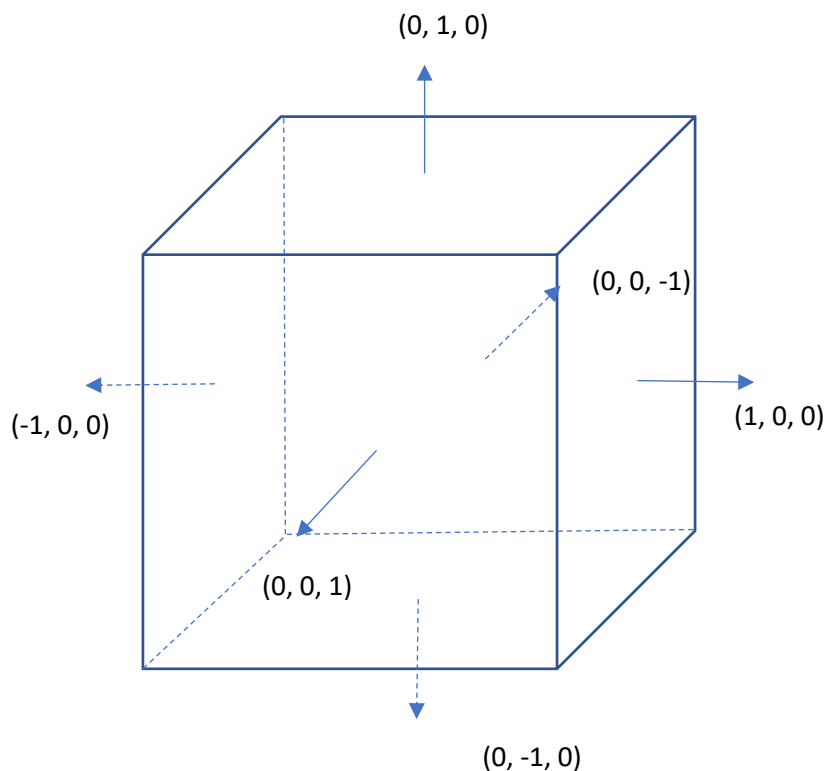
There are 27 cubes, which are arranged in a 3x3x3 grid. You can use the ModelViewerCamera to move the camera around. There is no light or colour yet, so all of the faces of each cube are white. In this lab we will add lights to this scene.

## Setup the vertex normal attributes

Each vertex, of each triangle, on each face, needs to have a normal attribute specified in the vertex array.

```
float vertices[] =  
{  
    //back face  
    //pos          //col          //normal  
    -0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    -0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    -0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
  
    //front face  
    -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    -0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
}
```

Each of the faces need to have a normal vector as follows, and each vertex on that face has the same normal vector.



Write the code in the vertex array in Lab9.cpp for each vertex normal.

Next, setup the VAO so that the vertex shader interprets the vertex attributes properly

```
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);
```

## Setup the uniforms

Create a variable to store the light direction vector

```
15
16 //light direction variable here
17 glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);
18
19 SCamera Camera;
~~
```

In the render loop just before the code for creating the view matrix, copy the vectors to uniforms in the shaders, i) light direction vector, ii) light colour vector, and iii) camera position vector. Note that these uniforms haven't been declared in the shaders yet, we will do that in the next section.

```
176 glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
177
178 glUniform3f(glGetUniformLocation(shaderProgram, "lightDirection"), lightDirection.x, lightDirection.y, lightDirection.z);
179 glUniform3f(glGetUniformLocation(shaderProgram, "lightColour"), 0.f, 0.f, 1.f);
180 glUniform3f(glGetUniformLocation(shaderProgram, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Position.z);
181
182 glm::mat4 view = glm::mat4(1.f);
183 view = glm::lookAt(Camera.Position, Camera.Position + Camera.Front, Camera.Up);
```

## In the vertex shader

In the vertex shader, add a new input vertex attribute for the normal (see line 5 below), add a new output variable for the normal (see line 12 below).

In the rendering pipeline, once the fragment shader has been called, the fragment position is in viewport space. We will need to use the fragment position in world space in the fragment shader to calculate the illumination. So, add an output variable for the fragment position (line 13).

In the shader main() function, transform the vertex position to world space using the model matrix, (line 18).

Also in main(), copy the vertex normal to the output variable (line 20). The extra transformations that are happening to the model matrix before it is multiplied with the vertex normal are to ensure that after any non-uniform scaling (i.e. a scale where x, y or z are different) of the model matrix, the normal remains perpendicular to the surface. This is beyond COMP3069 and you don't need to know about it, for now, use the code on line 20.

```
1  #version 330 core
2
3  layout(location = 0) in vec3 aPos;
4  layout(location = 1) in vec3 aCol;
5  layout(location = 2) in vec3 aNor;
6
7  uniform mat4 model;
8  uniform mat4 view;
9  uniform mat4 projection;
10
11  out vec3 col;
12  out vec3 nor;
13  out vec3 FragPos;
14
15  void main()
16  {
17      gl_Position = projection * view * model * vec4(aPos, 1.f);
18      FragPos = vec3(model * vec4(aPos, 1.f));
19      col = aCol;
20      nor = mat3(transpose(inverse(model))) * aNor;
21  }
22
```

# In the fragment shader

Create 2 new input variables to match the vertex shader output variable for the normal and output variable for the fragment position.

Add three vec3 uniforms, i) for the light colour, ii) for the light direction, and iii) for the camera position, so that the variable names match the code you added to the render loop earlier.

```
1  #version 330 core
2
3  in vec3 col;
4  in vec3 nor;
5  in vec3 FragPos;
6
7  uniform vec3 lightColour;
8  uniform vec3 lightDirection;
9  uniform vec3 camPos;
10
11 out vec4 fragColour;
12
13 void main()
14 {
```

Then, in main(), implement the direction light as it is defined in the lecture notes on page 5 – 6 and the pseudocode below.

```

ambient = k

Nnor = Normalise surface normal input
Nto_light = Negate and Normalise light direction uniform
Perform dot product Nnor with Nto_light, and ensure the result is positive
diffuse = result of dot product

Nfrom_light = Normalise light direction uniform
NrefLight = Reflect Nfrom_light on Nnor
camDirection = camera position uniform - fragment position input variable
NcamDirection = Normalise camDirection
Perform dot product of NcamDirection with rLight, ensure result is positive,
raise to the power of shininess
specular = result of dot product

phong = ambient + diffuse + specular

```

Note that the phong, ambient, diffuse and specular terms are all floating point variables.

When implementing ambient, use 0.1 as the constant.

When implementing diffuse, remember that the light direction is pointing from the light, so it is negated so that it is pointing towards the light.

When implementing specular, use 128 for the shininess value.

Some useful GLSL functions are

normalise using the glsl function `normalize()`

reflect using the glsl function `reflect()`

raise a number to a power using the glsl function `pow()`

Finally, at the bottom of `main()` in the fragment shader, use the formula for calculating the colour of a fragment from the lecture notes.

```

34 //fragColour = vec4(col, 1.f);
35 float phong = CalculateDirectionalIllumination();
36 fragColour = vec4(phong * col * lightColour, 1.f);
37 }

```

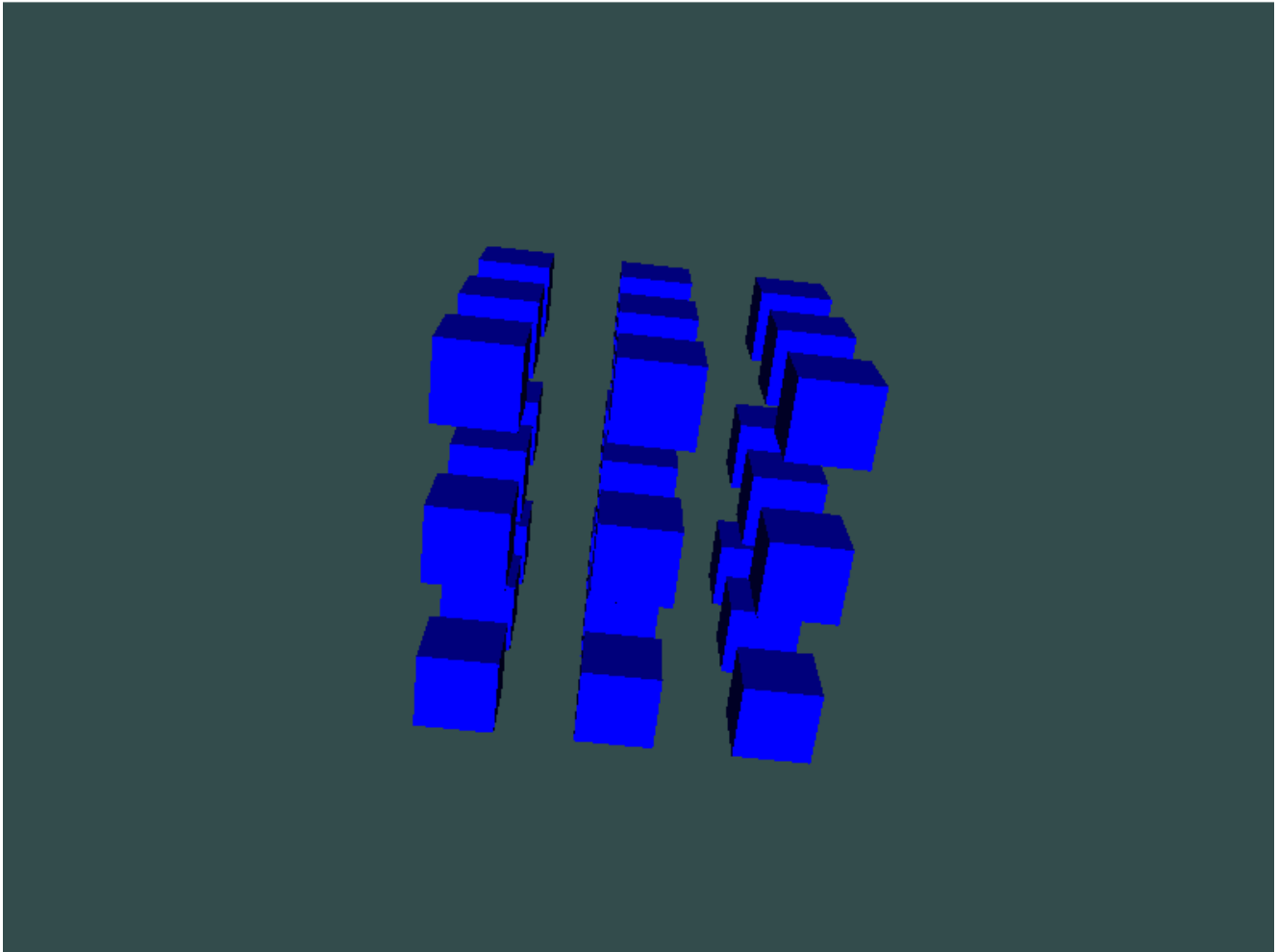
## Reset the light direction

Write some code in the `processKeyboard()` to reset the light direction to point in the same direction the camera is currently pointing

```
81 void processKeyboard(GLFWwindow* window)
82 {
83     glfwPollEvents();
84
85     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
86         glfwSetWindowShouldClose(window, true);
87
88     if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS)
89     {
90         lightDirection = Camera.Front;
91     }
92
93     bool cam_changed = false;
94     float x = 0.f, y = 0.f;
95     if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
96     {
```

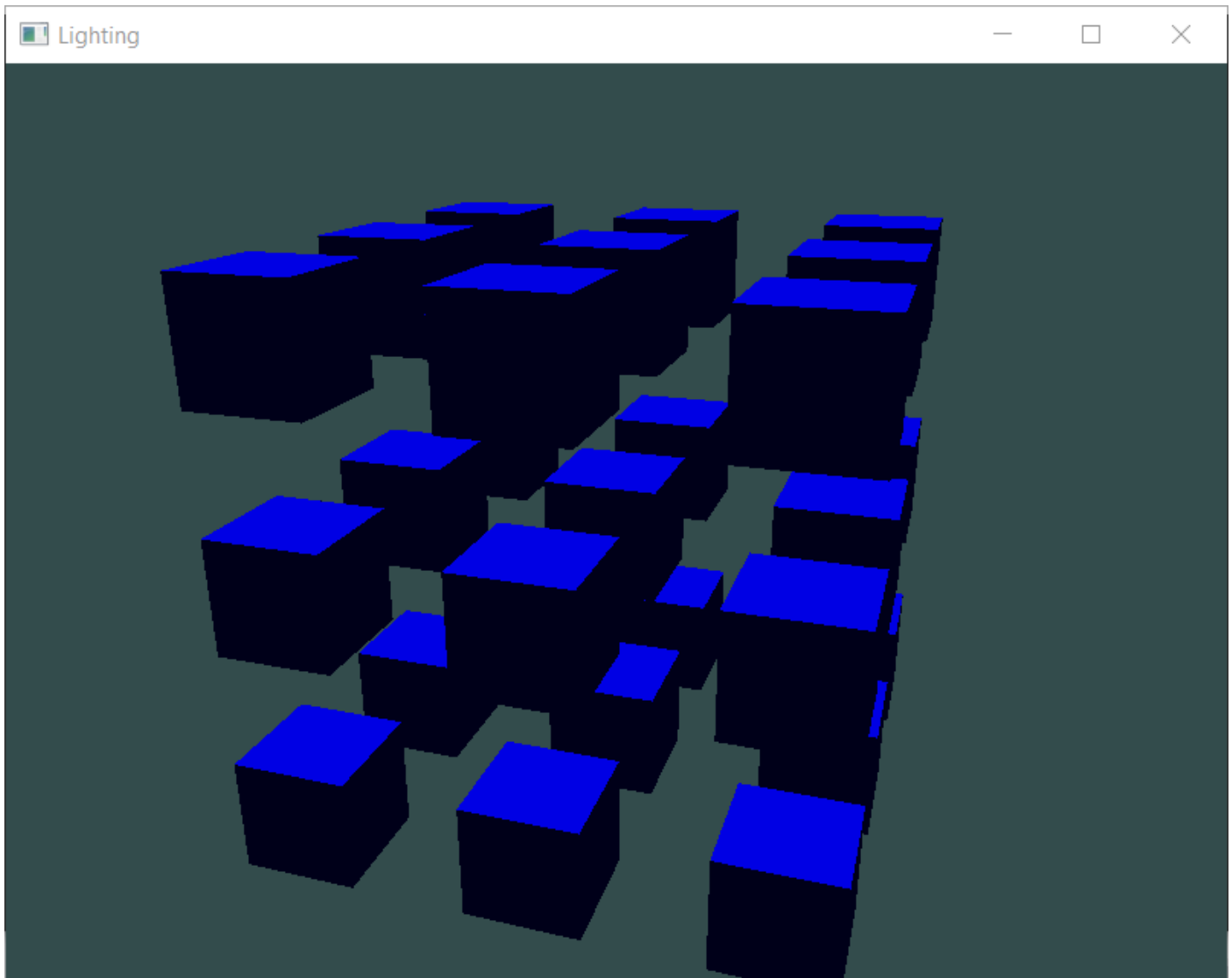
If you run the program, you should see this

Lighting





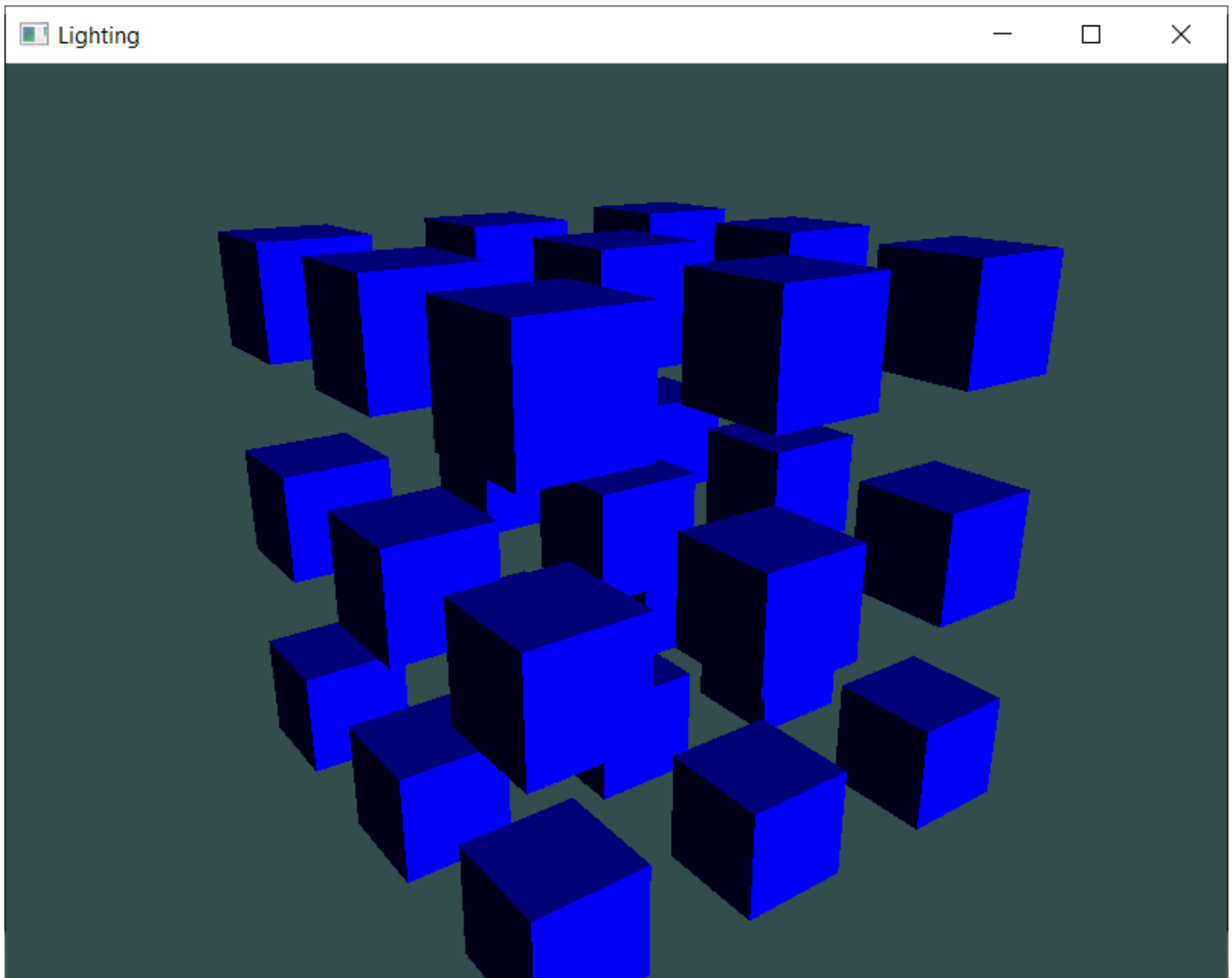
Move the camera up and back to see this



If you don't, check the initialisation of the lightDirection variable

```
15  
16 //light direction variable here  
17 glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);  
18  
19 SCamera Camera;  
20
```

If you press the space bar, then the directional light will point in the direction of the camera. Try it out and move the camera afterwards to a different position to see the effect of the directional light.



Notice how all of the fragments facing the same direction are illuminated exactly the same regardless of their position in the scene, because a directional light does not have a position.

Congratulations! You have now implemented some quite sophisticated lighting code in the fragment shader and have demonstrated the ability to program per fragment lighting.

**Demo – show the program to a member of the teaching team.**

## Add a function to the shader

Note that we have declared a function in the fragment shader, directly above `main()`, to keep the code tidy.

```
float CalculateDirectionalIllumination()
```

We have also moved the code for the directional light calculations from `main()` to the new function, and return the phong illumination value, then use it in the `main()` function.

```
28 void main()
29 {
30     float phong = CalculateDirectionalIllumination();
31
32     fragColour = vec4(phong * col * lightColour, 1.f);
33 }
34
```

Make sure the directional lighting calculations work correctly on each fragment.

## Change the light to a Positional Light

Now we are going to change the directional light to a positional light.

Start by adding a new function to the fragment shader just above `main()`

```
float CalculatePositionalIllumination()
```

Copy and paste the code from `CalculateDirectionalIllumination()` to `CalculatePositionalIllumination()`.

Change the code in `CalculatePositionalIllumination()` according to the lecture notes on page 8 and the pseudocode below

```
Ambient is the same as Directional Light

Diffuse is the same as Directional Light except
NToLight = normalize(lightPos - FragPos)

Specular is the same as Directional Light except
NFromLight = -NToLight

calculate attenuation

phong = (ambient + diffuse + specular) * attenuation
```

When calculating diffuse, we are going to use a new uniform called `lightPos`, which we will declare a little later on.

When calculating attenuation, use the glsl function `length()` to calculate the distance between the light and the fragment, and set constant attenuation to 1.5, linear attenuation to 0.05, and quadratic attenuation to 0.02.

Make sure you use the attenuation value when calculating phong.

Add a new uniform for light position

```
7 uniform vec3 lightColour;
8 uniform vec3 lightDirection;
9 uniform vec3 lightPos;
10 uniform vec3 camPos;
11
```

Declare a variable in Lab9.cpp for storing the light position

```
16 //light direction variable here
17 glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);
18 glm::vec3 lightPos = glm::vec3(2.f, 6.f, 7.f);
19
--
```

Copy the light position to the shader in the render loop

```
184 glUniform3f(glGetUniformLocation(shaderProgram, "lightDirection"), lightDirection.x, lightDirection.y, lightDir
185 glUniform3f(glGetUniformLocation(shaderProgram, "lightPos"), lightPos.x, lightPos.y, lightPos.z);
186 glUniform3f(glGetUniformLocation(shaderProgram, "lightColour"), 0.f, 0.f, 1.f);
187 glUniform3f(glGetUniformLocation(shaderProgram, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Positi
--
```

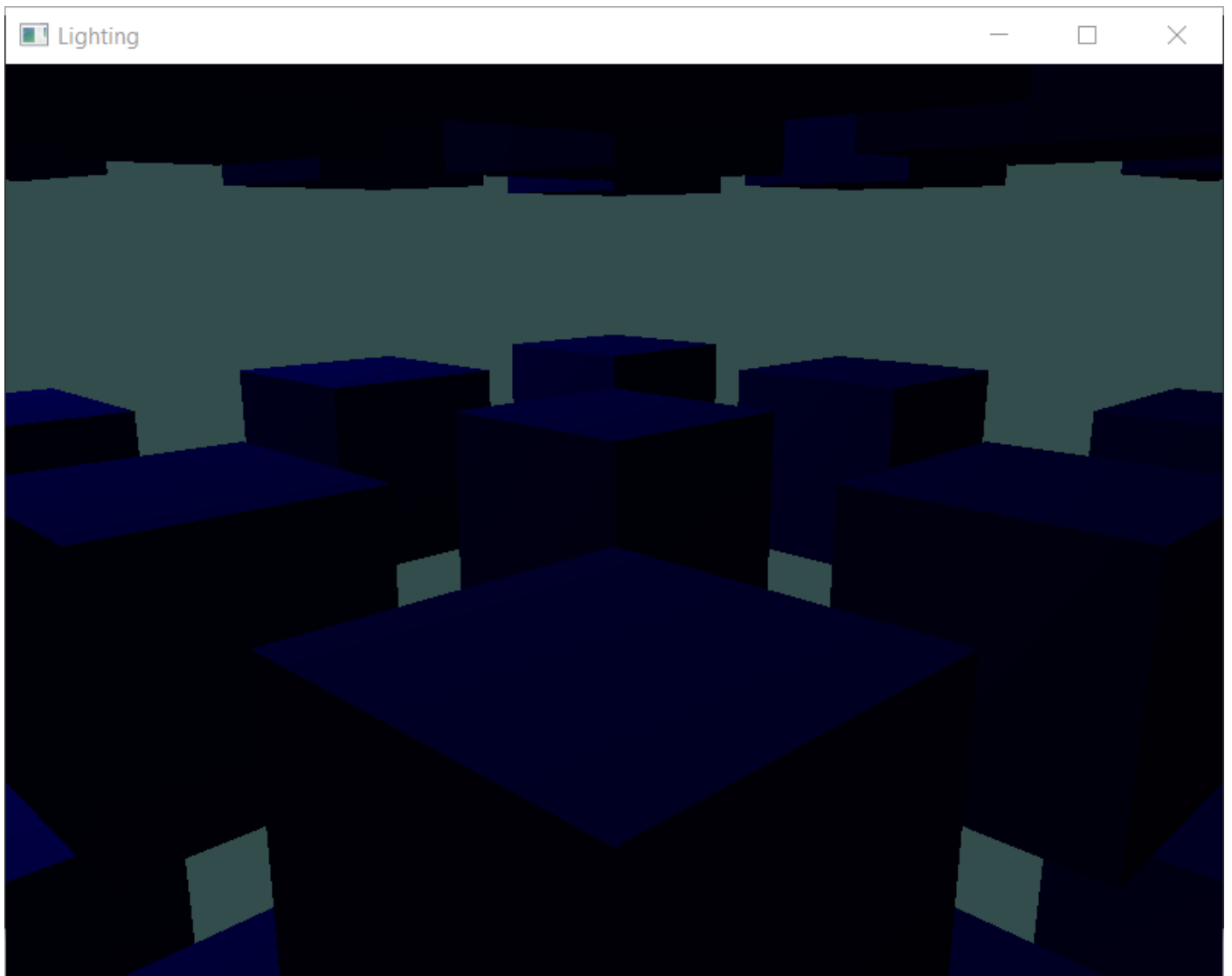
And in the processKeyboard() function set the light position to the camera position when the space bar is pressed

```
90 if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS)
91 {
92     lightDirection = Camera.Front;
93     lightPos = Camera.Position;
94 }
95
```

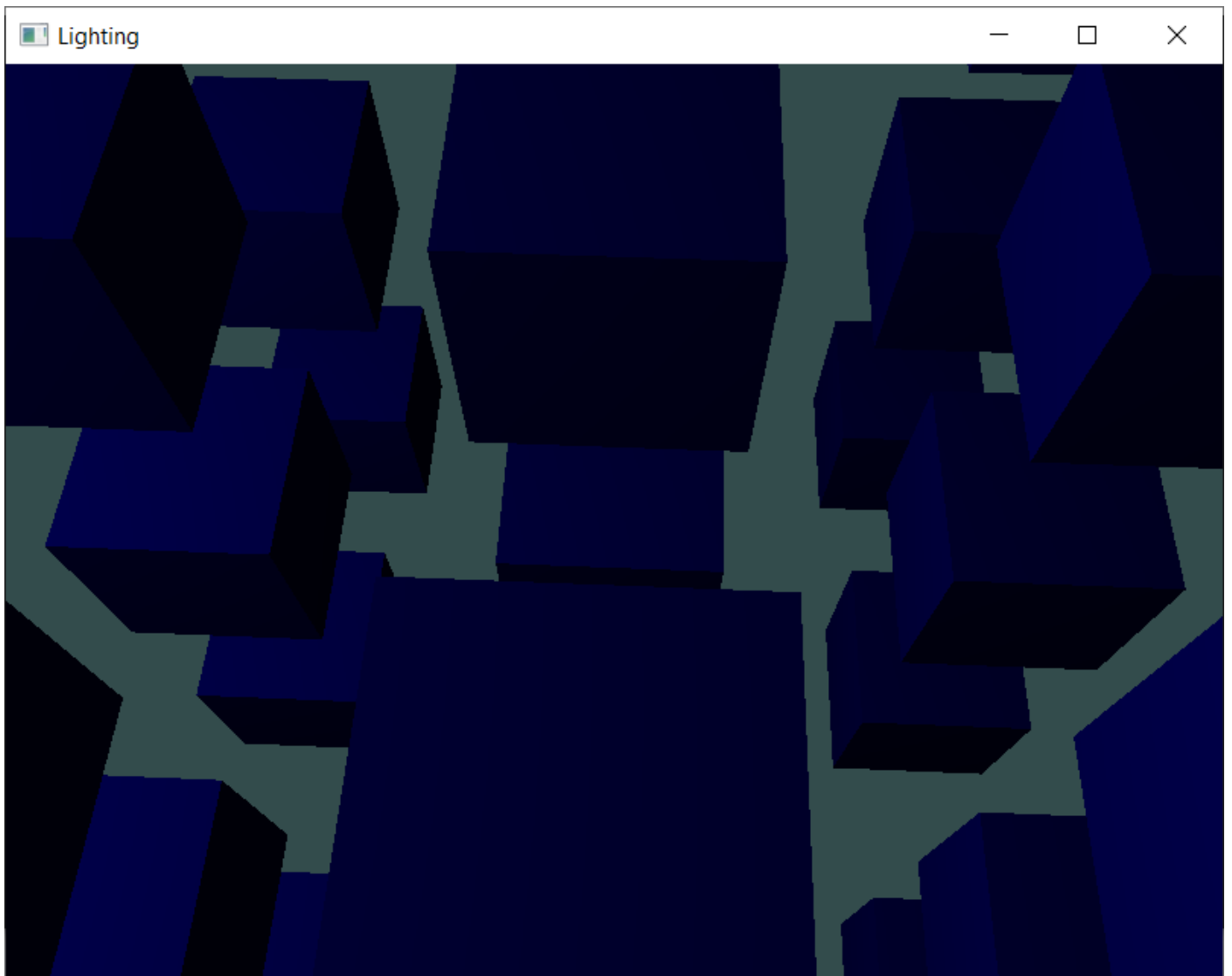
Finally, call the new function in your fragment shader to calculate the positional light illumination

```
50 void main()
51 {
52     float phong = CalculatePositionalIllumination();
53
54     fragColour = vec4(phong * col * lightColour, 1.f);
55 }
56
```

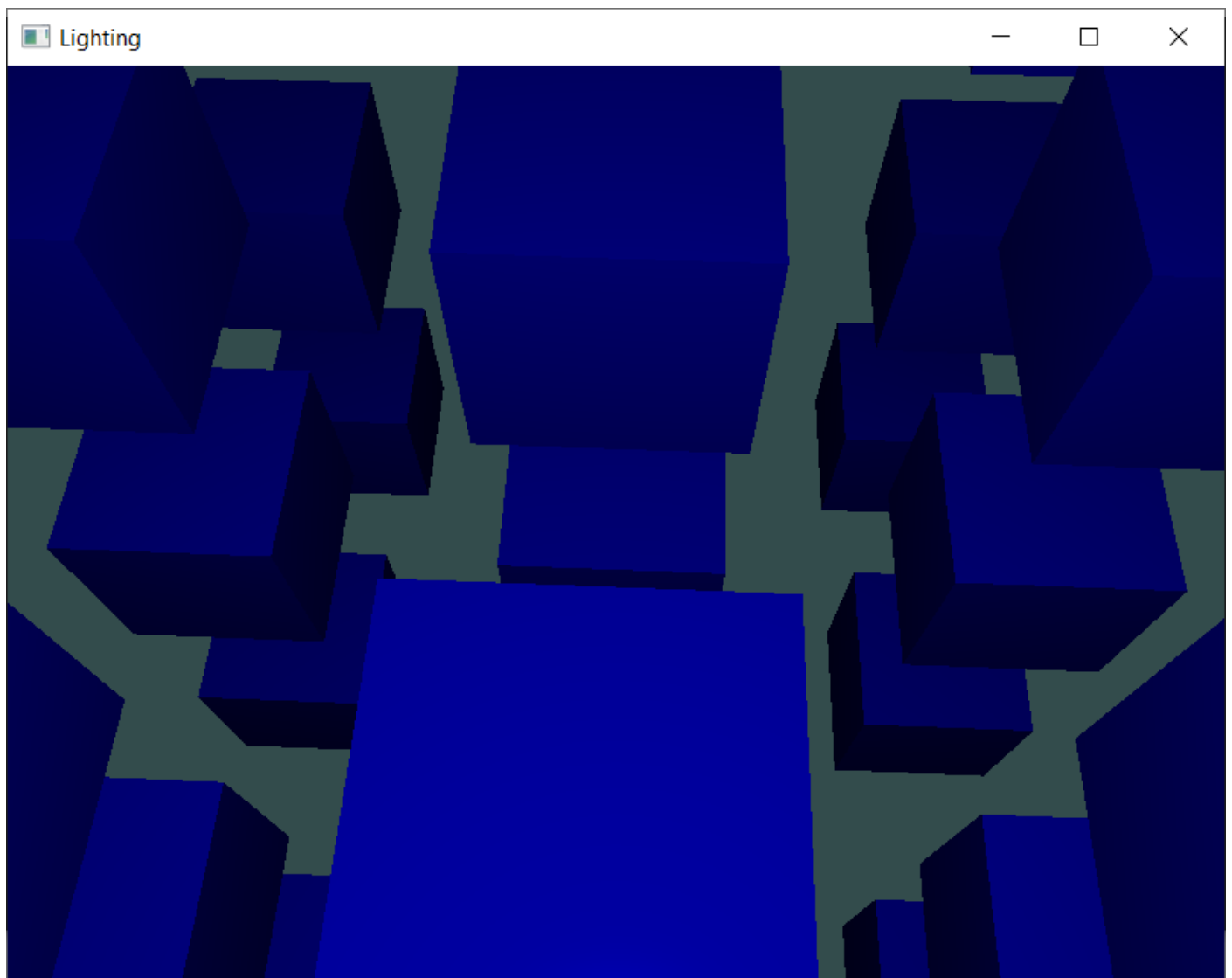
Run the program and you should see this



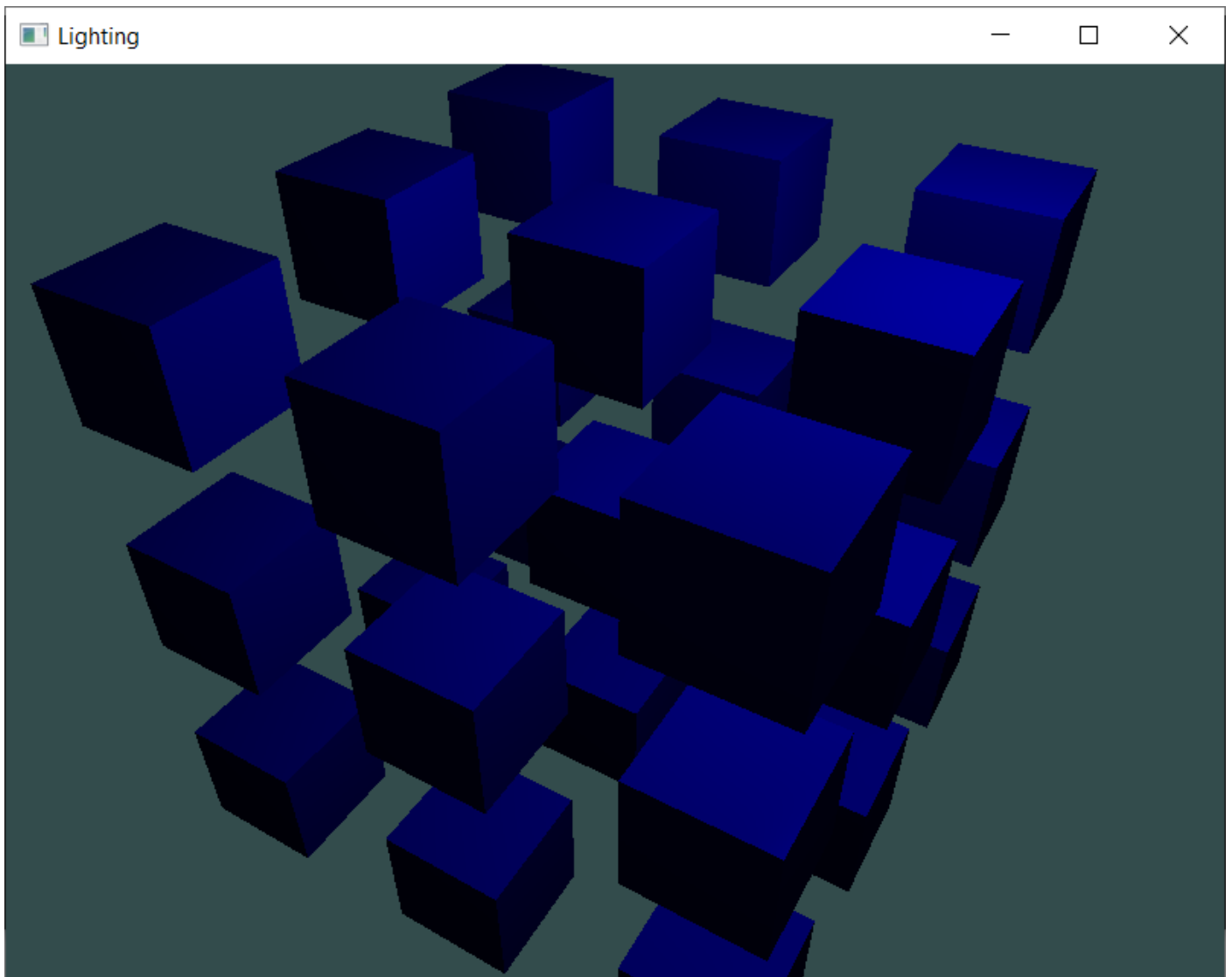
Move the camera to above one of the top row cubes



Press space bar to reposition the light (note that the light direction is still being set but not used in the positional light).



And then move the camera to a different position to see the effect of the positional light



Notice how the top face of the cube closest to where the light is positioned (where your camera was positioned when you pressed the space bar) is illuminated most brightly compared to the same faces on the other cubes. This is due to the attenuation.



# Change the light to a Spot Light

Now, we are going to change the positional light to a spot light.

Start by adding a new function to the fragment shader just above main()

```
float CalculateSpotIllumination()
```

Copy and paste the code from CalculatePositionalIllumination() to CalculateSpotIllumination().

Change the code in CalculateSpotIllumination() according to the lecture notes on page 11 and the pseudocode below

```
Ambient is the same as Positional Light

Diffuse is the same as Positional Light

Specular is the same as Positional Light

Attenuation is the same as Positional Light

phi = convert cut_off angle to radians and calculate cosine
NSpotDir = normalize lightDirection uniform
theta = dot product of NFromLight and NSpotDir

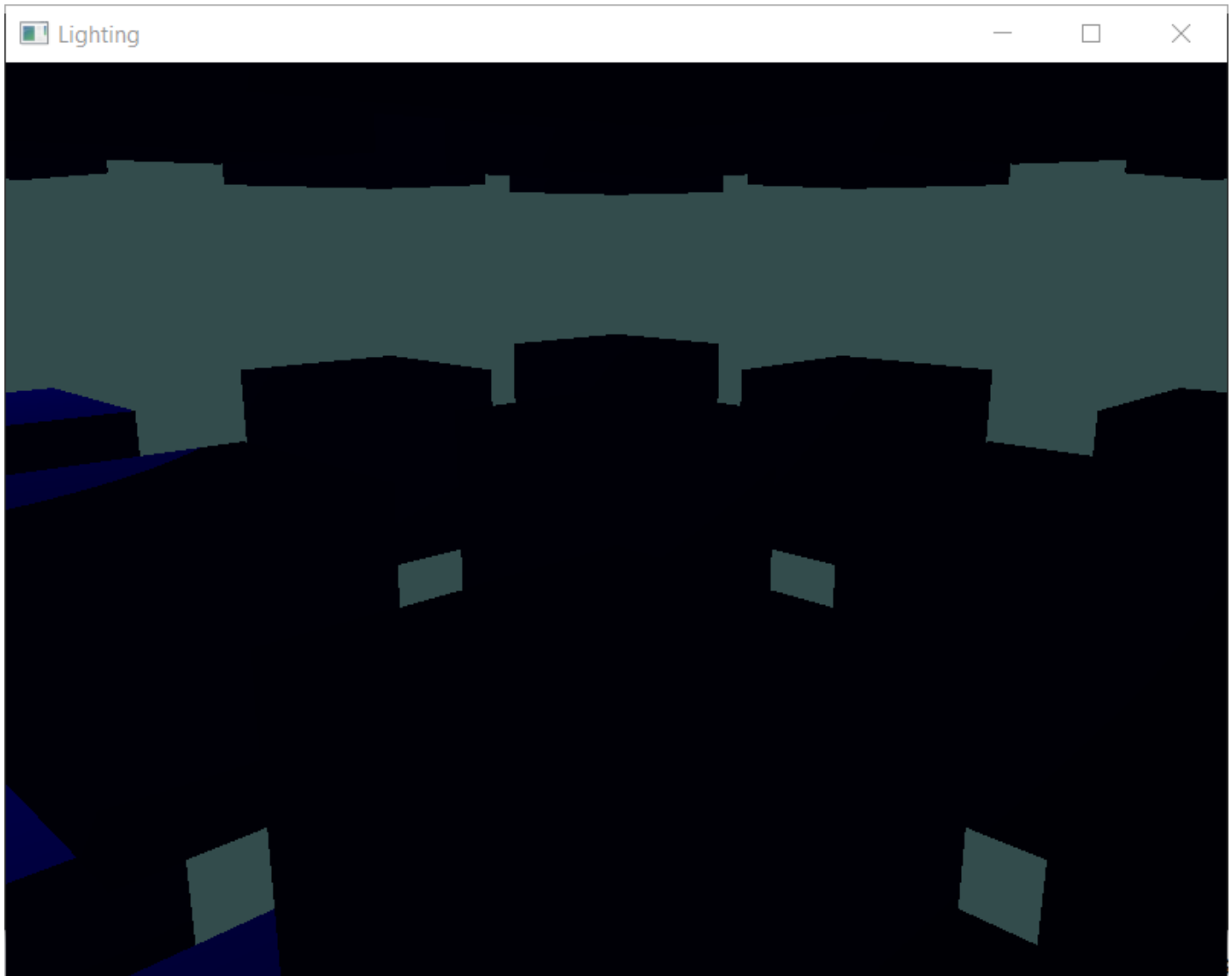
if(theta > phi)
{
    phong = (ambient + diffuse + specular) * attenuation
}
else
{
    phong = ambient * attenuation
}
```

When you calculate  $\phi$ , use the glsl functions `cos()` and `radians()` and a cut off angle of 15.

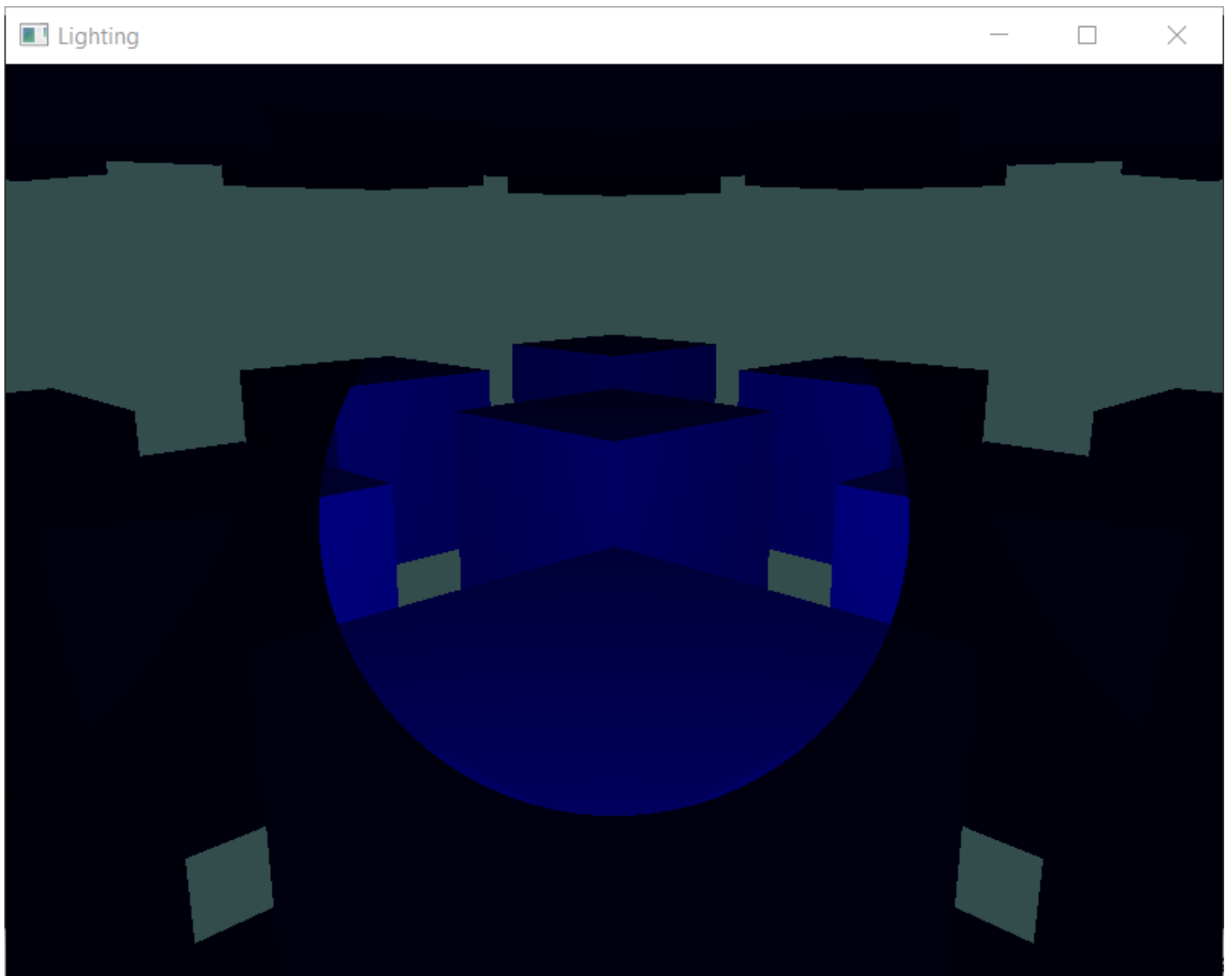
Finally, call the new function in your fragment shader to calculate the spot light illumination

```
89 void main()  
90 {  
91     float phong = CalculateSpotIllumination();  
92     fragColour = vec4(phong * col * lightColour, 1.f);  
93 }  
94  
95
```

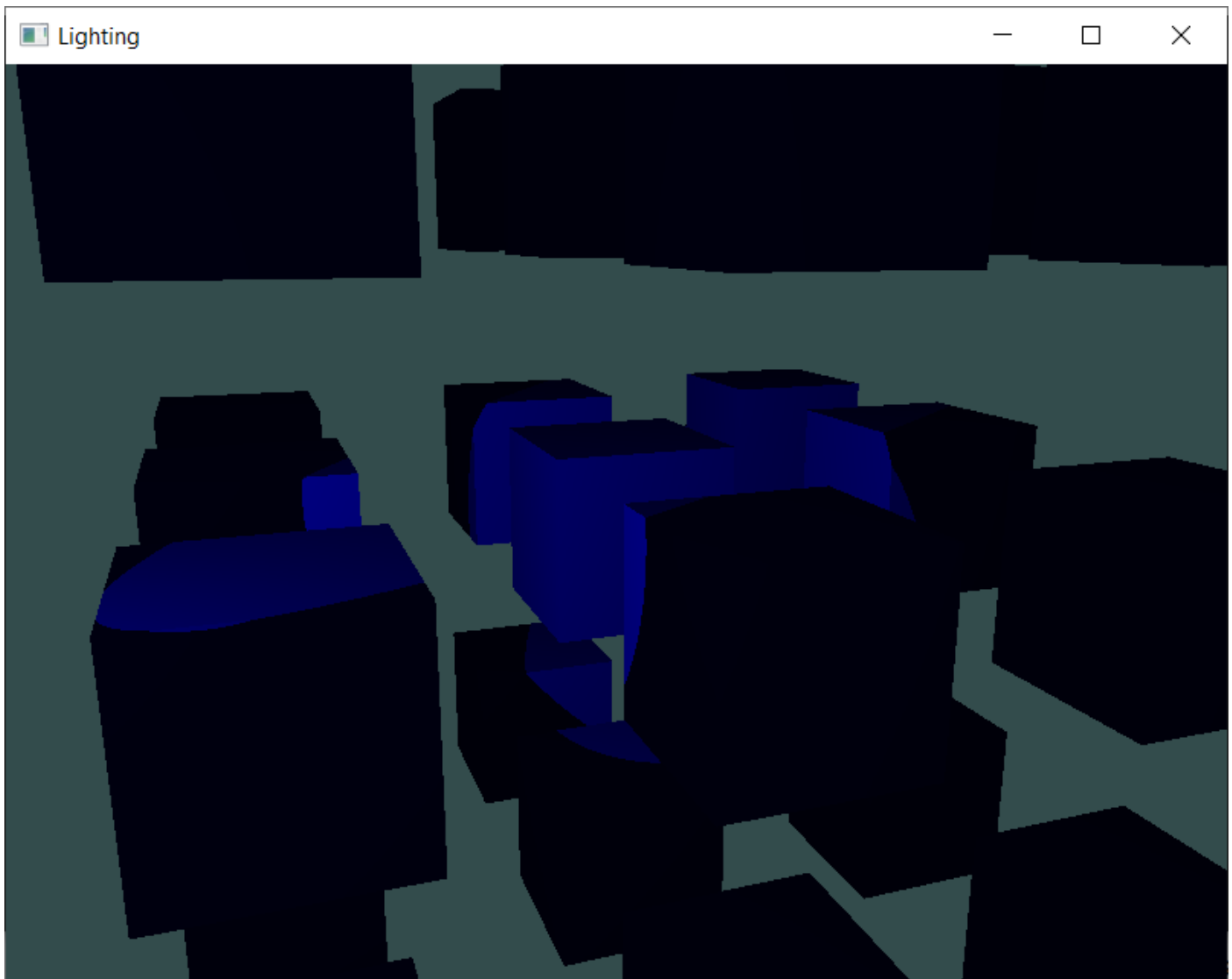
When you run the program, it should look like this



You can just make out the spot light illuminating some cubes on the left. Press space bar to re-position and direct the spot light so it matches the camera position and direction.



Then, move the camera to a new position to see the spot light effect.



## Remove duplicate code

Change the code in the fragment shader so there is no duplicate code. Defining a function

```
float CalculateAttenuation()
```

would probably be a good start.